Universidad Nacional Autónoma de México
Faculad de Ingeniería
Computer Engineering

Compilers

Lexer

Team: 04

Barco Núñez Claudia Citlali 422067621
Caballero Antunez Jesús Yael 320231364
Jacinto Robledo Valeria Berenice 320057973
Ospino Mérida Sebastián Emilio 319573471

Group: 05
Semester 2025-2

México, CDMX. March 2025

Contents

1 Introduction **2**
1.1 Goals 2

2 Theoretical Framework **3**

3 Development **5**
3.1 Notes on the lexer 6
3.1.1 Description of Data Types 6
3.2 Implementation of the grammar 8
3.3 Structure 8
3.4 Features 9

4 Results **9**
4.1 Tokenization Process 9
4.2 Output Analysis 12
4.3 Testing 13

5 Conclusions **16**

1 Introduction

The compilation process consist in a program in one language as an input and then translate into an equivalent program. The importance of the study of compilers relays in the ability to report report any errors in the source program that it detects during the translation process. The compilation divides in two phases, the analysis and the synthesis. The objective of the first phase is to convert character streams into words and recognize its token type initial step to build a compiler, the workflow can be divided into Lexical Analyzer, Syntax Analyzer and Semantic Analyzer. The first division, Lexical Analyzer, is are based on the basic conceptual units of linguistic morphological study and works by processing the initial stream of characters of the source program. Then, in accordance with the established rules, groups the characters into significant strings called lexemes. After the output that identify the token name, an abstract symbol representing a kind of lexical unit, and the attribute value that is associate with, the data is complete for the next phase, the Syntax Analyzer.

In the present project, we aim to create a Lexical Analyzer that can generate an output with the classification and number of tokens given a input in the target language. The Lexical Analyzer is made for the C language implemented on Python.

As a tool to build a more robust lexer, we decided to use **Lark**, a modern parsing library that makes it easy to build and process grammars. It is particularly useful for creating interpreters, compilers, and natural language processors. The key features are:

- **Supports multiple grammars:** Compatible with EBNF, JSON, Python, and more.
- **Fast and efficient:** Uses advanced algorithms like Earley and LALR(1) for quick and accurate parsing.
- **Extensible:** Allows custom rules and token processing functions.
- **Seamless Python integration:** Works well with regular expressions and Python data structures.

Lark is known for its flexibility and efficiency when dealing with language structures. Lark enables a more structured and declarative approach, allowing for better maintainability and scalability of the lexer. Its ability to process context-free grammars efficiently makes it an excellent choice for parsing languages with intricate syntax, such as C.
Considering the rule for parsing C language, Lark is an excellent choice as it allows us to define structured parsing rules while also efficiently handling preprocessor, comments, and various language constructs directives.

1.1 Goals

1. **Develop a functional lexical analyzer:** Build a program capable of identifying, classifying, and counting basic tokens present in a source code or text string, applying the theoretical concepts covered in class or using libraries that allow lexical analysis. The use of the FLEX tool is strictly prohibited.

2. **Implement a token recognition system:**The lexical analyzer must properly recognize and classify at least the following types of tokens:
   - Keywords: Reserved words that have a specific meaning in the language and cannot be used as identifiers.
   - Identifiers: Names of variables or functions that follow a valid pattern.
   - Operators: Symbols used for arithmetic, logical, or assignment operations.
   - Constants: Numerical values or literal strings present in the code.
   - Punctuation: Characters such as parentheses, commas, semicolons, etc.
3. **Count and present tokens:**Implement a mechanism that allows counting a total of 40 tokens and adequately presenting their classification, showing the total amount of each type of token identified in the analyzed source code or string.
4. **Process multiple types of input:**Develop the lexical analyzer so that it can accept as input either a text string or a file to be analyzed, ensuring proper processing for both options.
5. **Generate a structured report of the lexical analysis:**The program must generate a detailed report that includes:
   - The classification of each token found.
   - The total count of identified tokens.
   - The explanation and justification of the chosen token naming convention.
6. **Define an initial context-free grammar:**Propose a context-free grammar that represents at least one reading example used in the lexer, applying right recursion and left factoring when necessary.
7. **Apply good software development practices:**Structure the application using an appropriate development architecture (inspired by mobile or web development projects), following good Software Engineering practices, and ensuring the code is understandable, modular, and maintainable.
8. **Properly document the project:**Prepare a report detailing the decisions made during the development of the lexical analyzer, as well as explanations of the conventions and token names used.

2 Theoretical Framework

The first step we took in the development of the grammar we were going to use was the following definition:

$$\begin{aligned}\mathcal{S} ::= \; & \mathcal{S}\mathcal{S} \mid \mathcal{A} \mid \mathcal{B} \mid \mathcal{C} \mid \mathcal{D} \mid \mathcal{E} \mid \mathcal{F} \mid \mathcal{G} \mid \mathcal{H} \mid \mathcal{I} \mid \mathcal{J} \\ & \mid \mathcal{K} \mid \mathcal{L} \mid \mathcal{M} \mid \mathcal{N} \mid \mathcal{O} \mid \mathcal{P} \mid \mathcal{Q} \mid \mathcal{R} \mid \mathcal{T} \mid \mathcal{U} \mid \mathcal{V} \\ & \mid \mathcal{W}\end{aligned}$$

where each non-terminal represents:

- $\mathcal{A}$: preprocessor directives

- $\mathcal{B}$: data types
- $\mathcal{C}$: control structures
- $\mathcal{D}$: loops
- $\mathcal{E}$: function directives
- $\mathcal{F}$: memory management function identifiers
- $\mathcal{G}$: type modifiers
- $\mathcal{H}$: structure management directives
- $\mathcal{I}$: custom identifiers
- $\mathcal{J}$: arithmetic operators
- $\mathcal{K}$: comparison operators
- $\mathcal{L}$: logical operators
- $\mathcal{M}$: bitwise operators
- $\mathcal{N}$: assignment operators
- $\mathcal{O}$: increment/decrement operators
- $\mathcal{P}$: punctuation symbols
- $\mathcal{Q}$: numeric constants
- $\mathcal{R}$: pointer identifiers
- $\mathcal{T}$: address identifiers
- $\mathcal{U}$: typecasting
- $\mathcal{V}$: strings
- $\mathcal{W}$: characters

Considering all these production rules, we can notice that not all of them produce more than terminal characters. For instance, the production rules that are meant to determine operators return terminal values when used. Therefore, we focused on the set of rules which don't satisfy this characteristic, such as:

Custom identifiers

The identifiers we can use to name functions, variables, structures, can't start with a digit or any special character except for underscores. After the fist character, it can be used digits.

$\mathcal{I} \quad ::= \quad a\mathcal{I}_1 \quad | \quad ... \quad | \quad z\mathcal{I}_1 \quad | \quad A\mathcal{I}_1 \quad | \quad ... \quad | \quad Z\mathcal{I}_1 \quad | \quad _\mathcal{I}_1 \quad |$

$\mathrm{I}_1 \quad ::= \quad \mathcal{I}_2\mathcal{I}_1 \quad | \quad \mathcal{I}_2$

$\mathrm{I}_2 \quad ::= \quad a \quad | \quad ... \quad | \quad z \quad | \quad A \quad | \quad Z \quad | \quad 0 \quad | \quad ... \quad | \quad 9 \quad | \quad _$

In this subsection we can also take into account pointer identifiers and address identifiers because the only difference is that pointer names start with an asterisk, and address names start with an ampersand.

$\mathcal{R} \quad ::= \quad *\mathcal{I}$

$\mathcal{T} \quad ::= \quad \&\mathcal{I}$

Numeric constants

We can also include integers an real numbers

$\mathcal{Q} \quad ::= \quad \mathcal{Q}_1 \quad | \quad \mathcal{Q}_1.\mathcal{Q}_1$

$\mathrm{Q}_1 \quad ::= \quad \mathcal{Q}_2\mathcal{Q}_1 \quad | \quad \mathcal{Q}_2$

$\mathrm{Q}_2 \quad ::= \quad 0 \quad | \quad ... \quad | \quad 9$

Strings and characters

$\mathcal{V} \quad ::= \quad "V_1"$

$\mathrm{V}_1 \quad ::= \quad \mathcal{W}_1\mathcal{V}_1 \quad | \quad \mathcal{W}_1$

$\mathcal{W} \quad ::= \quad 'W_1'$

$\mathrm{W}_1 \quad ::= \quad a \quad | \quad ... \quad | \quad z \quad | \quad A \quad | \quad ... \quad | \quad Z \quad | \quad 0 \quad | \quad ... \quad | \quad 9 \quad | \quad _ \quad | \quad ...$

Preprocessor directives

$\mathcal{A} \quad ::= \quad \#include\ \mathcal{V}_1 \quad | \quad \#define\ \mathcal{V}_1$

Final grammar

$\mathcal{S} \quad ::= \quad \mathcal{S}_1\mathcal{S} \quad | \quad \mathcal{S}_1$

$\mathrm{S}_1 \quad ::= \quad \mathcal{A} \quad | \quad \mathcal{B} \quad | \quad \mathcal{C} \quad | \quad \mathcal{D} \quad | \quad \mathcal{E} \quad | \quad \mathcal{F} \quad | \quad \mathcal{G} \quad | \quad \mathcal{H} \quad | \quad \mathcal{I} \quad | \quad \mathcal{J}$

$\quad | \quad \mathcal{K} \quad | \quad \mathcal{L} \quad | \quad \mathcal{M} \quad | \quad \mathcal{N} \quad | \quad \mathcal{O} \quad | \quad \mathcal{P} \quad | \quad \mathcal{Q} \quad | \quad \mathcal{R} \quad | \quad \mathcal{T} \quad | \quad \mathcal{U} \quad | \quad \mathcal{V}$

$\quad | \quad \mathcal{W}$

3 Development

This lexer makes use of the lark python library. This library is part of a parser toolkit. Since a parser works with CFG, the toolkit is more than sufficient to construct a lexer, which would only require a Regular Grammar as a minimum to be constructed.

3.1 Notes on the lexer

The nature of the toolkit, allows for some more complex tokenization, since more complex grammar rules can be established. Henceforth, the lexer can present the tokens with superior precision, since it can use more elements of the grammar to determine what type of token it is reading.
Also, we can also use the fact that we are using a CFG for a lexer to cleverly avoid ambiguity. Also, the tool allows the use of regex to define terminal elements, which simplifies the writing of the grammar. And we can again indicate how the tree for the CFG should be constructed to avoid collisions with multiple elements that could match more than one regex, by adding priority to elements of the grammar. In essence, this makes lark a very promising choice as the base for our lexer.
Lark is also quite fast, since it implements many algorithms for the creation of different types of both parsers and lexer.

- Earley parser which handles any grammar, even if it is ambiguous, at the cost of some speed.
- LALR(1) which is standard for parsing, and will most likely be used by us in the future when working in the development of the parser, giving us even more arguments for choosing lark.
- Basic lexer.
- Context and dynamic lexers are implemented by Earley and LALR(1) respectively, which allow for efficient lexical analysis.
- Complete lexer which tries every possible combination at the cost of a serious downgrade in performance.

3.1.1 Description of Data Types

At the tokenization, we used a different token definition in order to obtain a more structured approach. This consideration is going to make easier the next compilation steps. The classification was made with a precise parsing and compiler optimization as a goal.

- **Preprocessor Directives (PREPROCESSOR)**: Commands that are processed before compilation begins. These directives are used to include files, define constants, and set conditional compilation rules.
- **Data Types (DATATYPE.2)**: Fundamental elements defining the type of data stored in memory. They determine how much space a variable occupies and what kind of values it can hold.
- **Control Structures (CONTROL_STRUCTURE.2)**: Constructs that define the flow of execution in a program. These include conditional statements and branching mechanisms.
- **Loops (LOOP.2)**: Repetitive constructs that execute a block of code multiple times based on a condition. They facilitate iteration without redundant code.

- **Function Handling (FUNCTION.2)**: Keywords related to defining and managing functions. These facilitate modular programming by allowing code reuse and structured design.
- **Memory Management (MEMORY.2)**: Operations related to dynamic memory allocation and deallocation. These are crucial for efficient resource management in a program.
- **Type Modifiers (TYPE_MODIFIER.2)**: Keywords that modify the behavior of data types, such as defining storage duration, mutability, and optimization hints.
- **Structures and Typedefs (STRUCTURE.2)**: Constructs that enable user-defined data types. They allow grouping related data into logical units for better organization and efficiency.
- **Identifiers (IDENTIFIER.1)**: Names assigned to variables, functions, and other program elements. They provide a way to reference stored values and functions in the code.
- **Arithmetic Operators (ARITHMETIC_OPERATOR)**: Symbols that perform mathematical operations on numeric values, enabling calculations and expressions.
- **Comparison Operators (COMPARISON_OPERATOR)**: Operators used to compare values, returning boolean results that determine conditional execution.
- **Logical Operators (LOGICAL_OPERATOR)**: Operators that evaluate logical expressions, often used in decision-making and control flow.
- **Bitwise Operators (BITWISE_OPERATOR)**: Operators that manipulate individual bits of binary numbers, commonly used in low-level programming.
- **Assignment Operators (ASSIGNMENT_OPERATOR)**: Operators that assign values to variables, supporting both basic and compound assignment operations.
- **Increment and Decrement Operators (INCREMENT_OPERATOR)**: Operators that increase or decrease variable values by one, often used in loops and counters.
- **Pointers and Address Operators**: Special symbols that work with memory addresses.
  - **Pointer Operator (POINTER_OPERATOR)**: Indicates that a variable is a pointer, storing memory addresses instead of values.
  - **Address Operator (ADDRESS_OPERATOR)**: Retrieves the memory address of a variable, enabling direct memory access.
  - **Pointer Identifier (POINTER_IDENTIFIER)**: A pointer declaration that associates a pointer with a specific identifier.

  - **Address Identifier (ADDRESS_IDENTIFIER)**: An identifier that refers to the memory address of a variable.

- **Constants (CONSTANT)**: Fixed values that do not change during program execution, including both integer and floating-point numbers.
- **Type Casting (TYPE_CAST)**: Explicit conversion of a variable from one data type to another, ensuring compatibility and precision in computations.
- **Strings and Characters**: Textual representations of data in a program.
  - **String (STRING)**: A sequence of characters enclosed in double quotes.
  - **Character (CHAR)**: A single character enclosed in single quotes.

3.2 Implementation of the grammar

Taking advantage of the features of the toolkit, we can simplify the grammar defined in the theoretical section.
Making use of regex to match some expressions like the preprocesor statements or identifiers. Also, as mentioned, we can write the keywords directly and check first if the input matches with any of them. After that we check the other types of tokens, with this, even our simplified implementation of the grammar is more than sufficient to create an accurate lexer.
The only current cost is performance, however this will be solved during the development of the parser, since we can add context to both the lexer and the parser which would mean the analysis is done much more quickly.

3.3 Structure

The project is structured so as to make it modular.

- **lex.py** contains the main code that makes use of the other elements.
- **lexer.py** contains a function that works with the leker to tokenize some input.
- **reader.py** contains a functions for reading both from terminal and from a file.
- **user.py** contains functions that deal with the user and the flags than can be used with the program.
- **tests/** contains .c code that is used for testing purposes, and the program outputs.
- **grammar.lark** is the file the contains the grammar the lexer will be using to analyse the string lexically.

Each of the different files is also subdivided in functions to allow for quick and simple debugging and maintenance.

3.4 Features

The lexer is designed to be a CLI tool (taking inspiration from tools like FLEX) and allows the following features:

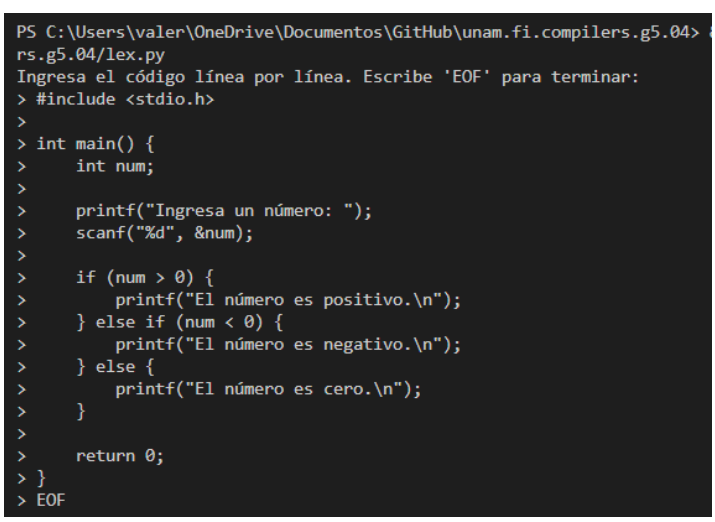
- Reading from the terminal until the user indicates they have reached the end of their input. This allows for quick testing of simple expressions, is practical in case the user does not want to create any files.
- Reading from a .c file. This allows for much complicated input strings, but gives the user freedom to analyse files they have written.
- Redirecting the output to a file. By default, the lexer will write the results of the tokenization to the terminal, but the user may indicate they want to save the output in a file. The program allows the user to do this, we recommend saving these output files with the extension **".yeyi"**.
- Printing a help text, which contains information about the options the program allows and how to use them.
- Printing the version of the program.

4 Results

4.1 Tokenization Process

The tokenization process begins with reading the input code, either from a file or entered manually through the terminal. The lexer then applies the defined grammar rules to break the source code into tokens, which are then displayed or saved to an output file if specified by the user.

- Source code from the terminal program **num.c**



```
PS C:\Users\valer\OneDrive\Documentos\GitHub\unam.fi.compilers.g5.04>
rs.g5.04/lex.py
Ingresa el código línea por línea. Escribe 'EOF' para terminar:
> #include <stdio.h>
>
> int main() {
>     int num;
>
>     printf("Ingresa un número: ");
>     scanf("%d", &num);
>
>     if (num > 0) {
>         printf("El número es positivo.\n");
>     } else if (num < 0) {
>         printf("El número es negativo.\n");
>     } else {
>         printf("El número es cero.\n");
>     }
>
>     return 0;
> }
> EOF
```

Figure 1: entered line by line.

```
Tokens encontrados:
PREPROCESSOR: #include <stdio.h>
DATATYPE: int
IDENTIFIER: main
PUNCTUATION: (
PUNCTUATION: )
PUNCTUATION: {
DATATYPE: int
IDENTIFIER: num
PUNCTUATION: ;
IDENTIFIER: printf
PUNCTUATION: (
STRING: "Ingresa un número: "
PUNCTUATION: )
PUNCTUATION: ;
IDENTIFIER: scanf
PUNCTUATION: (
STRING: "%d"
PUNCTUATION: ,
ADDRESS_IDENTIFIER: &num
PUNCTUATION: )
PUNCTUATION: ;
CONTROL_STRUCTURE: if
PUNCTUATION: (
IDENTIFIER: num
COMPARISON_OPERATOR: >
CONSTANT: 0
PUNCTUATION: )
PUNCTUATION: {
IDENTIFIER: printf
PUNCTUATION: (
STRING: "El número es positivo.\n"
PUNCTUATION: )
PUNCTUATION: ;
PUNCTUATION: }
CONTROL_STRUCTURE: else
CONTROL_STRUCTURE: if
PUNCTUATION: (
IDENTIFIER: num
COMPARISON_OPERATOR: <
CONSTANT: 0
PUNCTUATION: )
PUNCTUATION: {
IDENTIFIER: printf
PUNCTUATION: (
STRING: "El número es negativo.\n"
PUNCTUATION: )
PUNCTUATION: ;
PUNCTUATION: }
CONTROL_STRUCTURE: else
PUNCTUATION: {
IDENTIFIER: printf
PUNCTUATION: (
STRING: "El número es cero.\n"
PUNCTUATION: )
PUNCTUATION: ;
PUNCTUATION: }
FUNCTION: return
CONSTANT: 0
PUNCTUATION: ;
PUNCTUATION: }

Num tokens: 60
```

Figure 2: output in the terminal

- Source code from a file

```
PS C:\Users\valer\OneDrive\Documentos\GitHub\unam.fi.compilers.g5.04> python lex.py -f tests/use_cases/suma.c
```

Figure 3: suma.c

The output of **suma.c**:

```
Tokens encontrados:
PREPROCESSOR: #include <stdio.h>
DATATYPE: int
IDENTIFIER: main
PUNCTUATION: (
PUNCTUATION: )
PUNCTUATION: {
DATATYPE: int
IDENTIFIER: a
PUNCTUATION: ,
IDENTIFIER: b
PUNCTUATION: ,
IDENTIFIER: suma
PUNCTUATION: ;
IDENTIFIER: printf
PUNCTUATION: (
STRING: "Ingresa el primer número: "
PUNCTUATION: )
PUNCTUATION: ;
IDENTIFIER: scanf
PUNCTUATION: (
STRING: "%d"
PUNCTUATION: ,
ADDRESS_IDENTIFIER: &a
PUNCTUATION: )
PUNCTUATION: ;
IDENTIFIER: printf
PUNCTUATION: (
STRING: "Ingresa el segundo número: "
PUNCTUATION: )
PUNCTUATION: ;
IDENTIFIER: scanf
PUNCTUATION: (
STRING: "%d"
PUNCTUATION: ,
ADDRESS_IDENTIFIER: &b
PUNCTUATION: )
PUNCTUATION: ;
IDENTIFIER: suma
ASSIGNMENT_OPERATOR: =
IDENTIFIER: a
ARITHMETIC_OPERATOR: +
IDENTIFIER: b
PUNCTUATION: ;
IDENTIFIER: printf
PUNCTUATION: (
STRING: "La suma es: %d\n"
PUNCTUATION: ,
IDENTIFIER: suma
PUNCTUATION: )
PUNCTUATION: ;
FUNCTION: return
CONSTANT: 0
PUNCTUATION: ;
PUNCTUATION: }

Num tokens: 54
```

Figure 4: ”suma” output

4.2 Output Analysis

The tokens produced by the lexer are printed on the screen or saved to a file in the **tests/output/** directory. Each token is labeled with its type and value, along with the total number of tokens detected. The lexer proves to be efficient in processing input files and providing accurate results for each tested use case.

```
PS C:\Users\valer\OneDrive\Documentos\GitHub\unam.fi.compilers.g5.04> python lex.py -f tests/use_cases/memoria.c -o tests/output/memoria.yeyi
Salida guardada en tests/output/memoria.yeyi
PS C:\Users\valer\OneDrive\Documentos\GitHub\unam.fi.compilers.g5.04> python lex.py -f tests/use_cases/bits.c -o tests/output/bits.yeyi
Salida guardada en tests/output/bits.yeyi
PS C:\Users\valer\OneDrive\Documentos\GitHub\unam.fi.compilers.g5.04>
```

Figure 5: output files that are generated in the tests/output

Generated files:

- memoria.c

```
memoria.yeyi
tests > output > memoria.yeyi
PREPROCESSOR: #include <stdio.h>
PREPROCESSOR: #include <stdlib.h>
DATATYPE: int
IDENTIFIER: main
PUNCTUATION: (
PUNCTUATION: )
PUNCTUATION: {
DATATYPE: int
POINTER_IDENTIFIER: ****ptr
ASSIGNMENT_OPERATOR: =
TYPE_CAST: (int*)
MEMORY: malloc
PUNCTUATION: (
CONSTANT: 5
ARITHMETIC_OPERATOR: *
```

Figure 6: memoria.yeyi

- bit.c

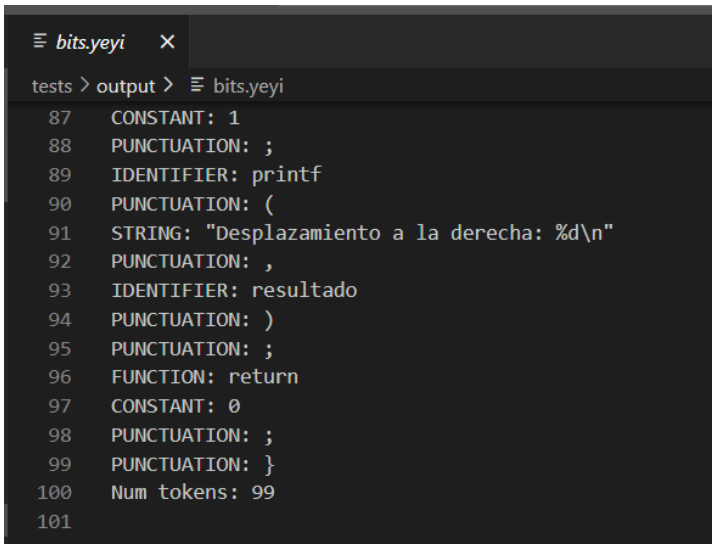


Figure 7: bit.yeyi

4.3 Testing

Testing was performed using various C-like code snippets located in the tests/ directory, such as num.c, suma.c, and bits.c. The output files generated during testing confirm that the lexer is functioning as intended, correctly identifying and categorizing each token present in the source files.
Below are some source codes:

suma.c | num.c | memoria.c | bits.c

tests > use_cases > suma.c > ...

```c
#include <stdio.h>

int main() {
    int a, b, suma;

    printf("Ingresa el primer número: ");
    scanf("%d", &a);

    printf("Ingresa el segundo número: ");
    scanf("%d", &b);

    suma = a + b;
    printf("La suma es: %d\n", suma);

    return 0;
}
```

Figure 8: suma.c

suma.c | num.c | memoria.c | bits.c

tests > use_cases > num.c > ...

```c
#include <stdio.h>

int main() {
    int num;

    printf("Ingresa un número: ");
    scanf("%d", &num);

    if (num > 0) {
        printf("El número es positivo.\n");
    } else if (num < 0) {
        printf("El número es negativo.\n");
    } else {
        printf("El número es cero.\n");
    }

    return 0;
}
```

Figure 9: num.c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int ****ptr = (int*) malloc(5 * sizeof(int));

    if (ptr == NULL) {
        printf("Error al asignar memoria.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        ptr[i] = i * 2;
        printf("Valor en ptr[%d] = %d\n", i, ptr[i]);
    }

    free(ptr);
    return 0;
}
```

Figure 10: memoria.c

```c
#include <stdio.h>

int main() {
    int a = 5, b = 3;
    int resultado;

    resultado = a & b;
    printf("AND: %d\n", resultado);

    resultado = a | b;
    printf("OR: %d\n", resultado);

    resultado = a ^ b;
    printf("XOR: %d\n", resultado);

    resultado = ~a;
    printf("NOT: %d\n", resultado);

    resultado = a << 1;
    printf("Desplazamiento a la izquierda: %d\n", resultado);

    resultado = a >> 1;
    printf("Desplazamiento a la derecha: %d\n", resultado);

    return 0;
}
```

Figure 11: bits.c

The lexer correctly identified keywords, identifiers, operators, numbers, and symbols. The tokens were saved to files located in the **tests/output/** directory as specified by the user. No errors were encountered during the processing of the files, demonstrating the robustness of the implementation.

5 Conclusions

Overall, the final version of the developed lexer was able to perform successfully. Thought the course of creating the lexer, we were able to fully understand the lexical analyzer and the relevance of a well-done token segmentation. In addition, we learned to work with the Lark tool, that achieve am efficient and modular implementation.

Moreover, the project achieved the listed objectives, enabling the tokenization and counting of any C language code. This accomplishment revealed the significance of having a well-structured lexical analyzer in larger systems, where the efficiency of token recognition and parsing can significantly impact performance.Through this process, we have gained a stronger foundation in compiler construction, which is a crucial aspect of programming language design.

References

[1] A. V. Ho, M. S. Lam, and R. Sethi, *Compilers: Principles, Techniques and Tools*, 2nd ed. Pearson Education, 2009.

[2] How To Use Lark - Guide — Lark documentation, Readthedocs.io, 2020. [Online]. Available: https://lark-parser.readthedocs.io/en/stable/how_to_use.html

[3] T. Pai and P. S. Aithal, A systematic literature review of lexical analyzer implementation techniques in compiler design, *International Journal of Applied Engineering and Management Letters (IJAEML)*, vol. 4, no. 2, pp. 285–301, 2020.